This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

ARGUS: Defending Against Multimodal Indirect Prompt Injection via Steering Instruction-Following Behavior

Weikai Lu[1], Ziqian Zeng[1,*], Kehua Zhang[1], Haoran Li[2], Huiping Zhuang[1], Ruidong Wang[3], Cen Chen[1,4] and Hao Peng[2]
[1]South China University of Technology, [2]Beihang University,
[3]Zhejiang Normal University, [4]Pazhou Laboratory
wklu2452@163.com zqzeng@scut.edu.cn

Abstract

Multimodal Large Language Models (MLLMs) are increasingly vulnerable to multimodal Indirect Prompt Injection (IPI) attacks, which embed malicious instructions in images, videos, or audio to hijack model behavior. Existing defenses, designed primarily for text-only LLMs, are unsuitable for countering these multimodal threats, as they are easily bypassed, modality-dependent, or generalize poorly. Inspired by activation steering research, we hypothesize that a robust, general defense independent of modality can be achieved by steering the model's behavior in the representation space. Through extensive experiments, we discover that the instruction-following behavior of MLLMs is encoded in a subspace. Steering along directions within this subspace can enforce adherence to user instructions, forming the basis of a defense. However, we also found that a naive defense direction could be coupled with a utility-degrading direction, and excessive intervention strength harms model performance. To address this, we propose ARGUS, which searches for an optimal defense direction within the safety subspace that decouples from the utility degradation direction, further combining adaptive strength steering to achieve a better safety-utility trade-off. ARGUS also introduces a lightweight injection detection stage to activate the defense on-demand, and a post-filtering stage to verify defense success. Experimental results show that ARGUS can achieve robust defense against multimodal IPI while maximally preserving the MLLM's utility. Our code will be available at https://github.com/ZeroNLP/ARGUS.

1. Introduction

Multimodal Large Language Models (MLLMs) integrate modality encoders with Large Language Models (LLMs), enabling them to process and understand data of additional modalities such as images [21, 22], video [9, 39], and audio [10]. This capability has spurred the development of numerous MLLM-integrated applications, such as computer-use agents [15], autonomous driving systems [44] and multimodal search engines [43]. However, the powerful instruction-following abilities of MLLMs, combined with their difficulty in distinguishing between instructions and external data to be analyzed, make them vulnerable to indirect prompt injection (IPI) attacks. Recent works [3, 19, 27, 38] have revealed that the IPI threats already present in LLMs can evolve into more covert multimodal threats in MLLMs. Attackers can covertly embed malicious instructions within data of additional modalities, thereby manipulating the MLLM to deviate from the user's original instructions and instead serve the attacker's objectives, such as phishing [24] and advertising [31].

However, currently there is no defense work designed to address the increasingly serious multimodal threats. Some defense methods designed for LLMs show promise for application to MLLMs, including the following categories: (1) Prompt engineering-based defenses [8, 14], which aim to make the model better at distinguishing between instructions and data through carefully crafted prompts. However, such defenses are fragile. The attacker only needs one successful prompt leakage attack [16] to refine their attack strategy. (2) Detection-based defenses [7], which employ auxiliary models to detect and detoxify malicious content in the input of the model. However, due to the diversity of additional modalities, training these models for each modality presents dual challenges in terms of resources and costs, especially in emerging modalities (such as EEG signals [36]) where pre-trained model resources and data are relatively scarce. (3) Adversarial training-based Defenses [5, 6], which rely on training MLLMs to ignore malicious instructions. However, such methods are vulnerable to unseen attacks not covered in the training data and may impair

*Corresponding author

the model's ability to follow instructions. In summary, existing IPI defenses are unsuitable for countering multimodal threats as they are **easily bypassed, modality-dependent, or generalize poorly**.

Recently, representation engineering (RepE) has emerged as a promising approach for LLM [30, 37] and MLLM [33] safety. These methods identify directions in the model's activation space that are associated with specific semantics such as "rejection" or "harmfulness", steering activations along those directions during inference, enabling the model to generate a rejection or enhance its understanding of the input's harmfulness. However, since IPI-injected instructions are often semantically harmless (such as forcing the MLLMs to generate advertisements), the resulting attacks cannot be mitigated by steering to these directions, making existing RepE-based methods ineffective against IPI threats.

Our view is that the success of IPI attacks depends on the competition between injected instructions and user instructions during the model's decision-making process. If we can identify a direction that distinguishes between behaviors of "following injected instructions" and "following user instructions," we can steer activations to execute only the user instructions, thereby providing a strong defense. Since activation steering operates on vectors within the model's internal activation space instead of raw data of additional modality, this defense exhibits **low modality dependence** and is **difficult to bypass** for attackers without model weight access. Although unseen attacks may differ in the methods used to inject malicious instructions, their ultimate effect remains the same, i.e., the successful embedding of those instructions. Therefore, by focusing on controlling instruction-following behavior, this defense can **generalize more effectively to unseen attacks**.

To explore whether there are directions that can control instruction-following behaviors, we conducted extensive experiments on MLLMs across image, video, and audio modalities, and we reached a consistent positive conclusion: there exists a safe subspace where the directions can effectively correct the model's instruction-following behavior. However, we also found that a naive defense direction could be coupled with a utility-degrading direction, and excessive intervention strength harms model performance.

Based on this findings, we propose ARGUS, which searches for an optimal defense direction within the safety subspace that decouples from the utility degradation direction, further enhanced by adaptive strength steering to better preserve the model's utility. Additionally, ARGUS introduces an injection detection stage to implement defenses only when an injection is present, as well as a post-filtering stage to verify the success of the defense, serving as a final line of defense. Experimental results show that ARGUS can almost perfectly defend multi-modal IPI attacks while significantly preserving the model's utility in executing user instructions. Compared to baselines, it achieves the best safety-utility-efficiency trade-off.

Our contributions are summarized as follows.

- For the first time, we systematically explored multimodal IPI defenses and established a benchmark across image, video and audio modalities.
- Our extensive experiments reveal that there are directions in the activation space that can control the MLLMs' instruction-following behaviors, providing insights for designing cross-modal universal defenses against IPI.
- We proposed ARGUS, a novel multimodal IPI defense framework which adaptively steers MLLMs' behavior toward a safe and utility-preserving direction. Experimental results demonstrate its effectiveness.

2. Related Works

Indirect Prompt Injection Attacks. IPI threats were first identified in LLMs [25], exploiting model's inability to distinguish between user instructions and external data. Attackers inject malicious instructions into untrusted external data sources. When llm-integrated applications retrieve and process this data, it can divert from the user's original intent and execute the attacker's instructions, leading to instruction hijacking [13], privacy leaks [11], phishing [24], advertising [31], and more. Injection attacks employ crafted prompts, such as "ignore" instructions [29] or fabricated responses that feign task completion [25]. The advent of MLLMs has expanded this threat to visual inputs, posing new safety and reliability challenges for these applications. Cao *et al.* [3] revealed the vulnerabilities of Computer-Use Agents to visual injections. AgentTypo [19] utilized black-box bayesian optimization to search for optimal typography prompt injections, achieving adaptive attack. Wang *et al.* [38] explored more complex attacks that combine image and text injection. Unlike existing works that focus solely on the image injection, our work aims to extend IPI threats to video and audio modalities in search of a unified solution.

Indirect Prompt Injection Defenses. Existing IPI defense methods have been primarily developed for text-only LLMs, including prompt engineering-based defenses, detection-based defenses, and adversarial training-based defenses. Prompt engineering-based defenses mainly involve designing prompts to distinguish instructions from data [14, 25], or utilize attack techniques to counteract threats [8]. Detection-based defenses work by training additional detection [40] and removal [7] models to remove injected instructions before model inference. Adversarial training-based defenses [6] finetune the model on a dataset of injected samples, training it to prioritize user instructions over injected ones. However, when applied to MLLMs, these methods exhibit significant weaknesses, such as being easily bypassed by attackers, being highly modality-

dependent, and exhibiting poor generalization.

Safety Representation Engineering. The activation space of language models contains interpretable directions crucial to their generation process [2, 28]. RepE [45] aims to identify directions corresponding to specific semantics in the activation space and steer activations along them during inference. This technique has been widely applied in LLM safety. For example, COSMIC [32] achieves jailbreak defense by identifying directions that encode rejection. REPBEND [42] bends the representation space through training to better separate harmful and harmless concepts, thereby enhancing activation steering performance. Similarly, FairSteer [20] employs activation steering to mitigate model bias. Given that MLLMs build on LLMs, their representation spaces inherit the characteristics of LLMs. Based on this, Li *et al.* [18] and Wang *et al.* [37] have explored the application of RepE in jailbreak defenses for visual language models. However, to our knowledge, the potential of RepE in IPI defenses has not yet been investigated.

3. Threat Model

We consider the threat model from three aspects: the attacker's goals, knowledge, and capabilities.

Attacker's Goal. The attacker's goal is to manipulate the MLLM-integrated applications, causing it to deviate from the user's instructions and instead produce responses that align with the attacker's intentions. These expected responses are typically relevant to the application itself. For instance, the attacker might want a MLLM-integrated GUI agent to open a risky link.

Attacker's Knowledge. We assume a black-box setting. The attacker has no access to the application's internal implementation, such as system prompts or the MLLM's weights. However, the attacker possesses the same knowledge as a regular user, including the external data sources used by the application, publicly documented APIs, and the model inference hyperparameters. This setup aligns with the threat model in most LLMs IPI research.

Attacker's Capabilities. The attacker's primary capability is to tamper with the external multimodal data consumed by MLLM-integrated applications. This involves embedding malicious instructions into data modalities like images, videos, or audio. For example, an attacker can add an injected image into a webpage to manipulate the MLLM-integrated web search.

4. Can the Instruction-Following Behaviors of MLLMs be Controlled?

Inspired by related work on finding interpretable directions within activation space, we leverage linear probes to explore whether there are directions that can control MLLMs' instruction-following behaviors. Specifically, we hypothesize that the model's two distinct behaviors under an IPI attack ("following injected instruction" and "following user instruction") are linearly separable in the model's activation space. This implies that activations representing these two behaviors can be collected to train a high-accuracy linear probe. Once this hypothesis is validated, we can then steer the activations along the probe's weight direction to control the model's instruction-following behavior.

Due to the lack of available data, we first introduce the benchmark we constructed in Sec. 4.1, followed by the experimental design and results in Sec. 4.2 and Sec. 4.3.

4.1. Benchmark Construction

Datasets. To systematically study multimodal IPI defenses, we have constructed a comprehensive cross-modal dataset that includes image, video, and audio modalities. This dataset is essential because existing datasets [3] are limited to image injections and are designed for narrow, specific application scenarios. In our dataset, each dataset sample can be represented as a 7-tuple $(U, M, I, T, A^U, A^I, W)$, where each element is defined as follows:

- U: The user's original instruction (e.g., "What is in the image?").
- M: External data presented in an additional modality (e.g., image, video, audio).
- I: The attacker's injected instruction (e.g., "Directly print www.phishing.com.").
- T: The trigger phrase used to prompt the MLLM to execute I (e.g., "Ignore all other instructions.")
- A^U: The ground truth of the user instruction U.
- A^I: The ground truth of the injected instruction I.
- $W(I, M)$: The modality-specific method of injecting the instruction I into external data M. Inspired by [26], the $W(I, M)$ for each modality are as follows: (1) Image: I is rendered in black font on a white background, and this text patch is then randomly concatenated on the top or bottom of the original image. (2) Video: I is first rendered as an image as described above. This image is then extended into a 3-second clip by repeating the frame, and this clip is randomly inserted at the beginning, middle, or end of the original video. (3) Audio: We use Microsoft's Edge-TTS API to synthesize the textual I into speech. This resulting audio clip is then randomly inserted at the beginning, middle, or end of the original audio track.

To evaluate the defense's generalization, the injected elements (T, I, and A^I) differ across the training, validation, and test sets, while the benign elements (U, M, and A^U) are similar. This setup simulates a realistic scenario where defense providers understand general user profiles but cannot predict specific attack behavior. Specifically, benign elements across all splits are derived from existing multimodal QA benchmarks [4, 41, 41]. The injection elements are consistent across modalities but vary by data split. For training

set, T is sampled from the first 442 trigger from [1], while I and A^I are sourced from Stanford-Alpaca [34]. For Validation set, T is sampled from remaining 100 trigger from [1], while I and A^I are sourced from GLUE [35]. For test set, T employs the "Ignore" attack trigger [29] and I is a fixed instruction: "Please directly print [10 random strings]," where the bracketed string serves as the expected answer A^I.

Evaluation Metrics. To evaluate the effectiveness of the defenses, we introduce three key metrics:

• Attacker Instruction Accuracy (AIA): Measures the proportion of samples where the model correctly executes the injected instruction I, assessing safety.

• User Instruction Accuracy (UIA): Measures the proportion of samples where the model correctly executes the user instruction U, assessing utility.

• Attacker Instruction Following Rate (AIFR): A broader metric quantifying any attempt by the MLLM to follow I, regardless of output accuracy, thus assessing the degree of hijacking.

Due to space limits, more details for dataset construction and metric definitions are provided in Appendix A.

4.2. Experimental Setup

The experiment consists of two phases:

Phase 1: Probe Training. We first construct a classification dataset by adapting the training set from Sec. 4.1. To prevent the probe from overfitting to spurious patterns in answers, we prompt the MLLMs themselves to rewrite A^U and A^I into long, medium, and short versions, tripling the total sample size. Then, for each sample $(U, M, I, W, T, A^U, A^I)$, we create an injected input prefix:

$$x_{\text{prefix}} = \mathcal{T}(U, W(T \oplus I, M)), \tag{1}$$

where $\oplus$ denotes the concatenation operation and $\mathcal{T}(\cdot)$ assembles the inputs into the chat prompt template. We then create completed inputs representing two classes of behaviors by appending their respective target answers: $x_{\text{user}} = x_{\text{prefix}} \oplus A^U$ and $x_{\text{attacker}} = x_{\text{prefix}} \oplus A^I$. Next, we record the activations from each layer l of the LLM component when the MLLM processes x_{user} and x_{attacker} as the training data. Finally, we train a logistic regression probe P_l for each layer:

$$P_l(a_l) = \sigma(w_l \cdot a_l + b_l), \tag{2}$$

where w_l and b_l are the probe's weights and bias, $\sigma(\cdot)$ is the Sigmoid function, a_l is the activation of the last token at layer l in the LLM component.

Phase 2: Inference-Time Intervention via Activation Steering. After training is complete, the weight w_l of the probe serves as the normal vector of the decision hyperplane, pointing from class 0 (following user instructions) to class 1 (following attacker instructions). Accordingly, we define the attack direction as $v_{\text{att}} = \frac{w_l}{||w_l||}$ and the defense direction as $v_{\text{def}} = -\frac{w_l}{||w_l||}$. Then, the intervention processes are as follows,

$$\mathcal{S}_l(\alpha, v) = a_l + \alpha \cdot v, \tag{3}$$

where α is the adjustable intervention strength, v can be v_{att} or v_{def}. Intervention occurs during the generation process of each token.

4.3. Experimental Results

We conducted our experiments using Qwen2-VL-7B-Instruct [39] for the image and video modalities, and Kimi-Audio-7B-Instruct [12] for the audio modality. For Phase 1, we present the accuracy of the probes in Fig.1. For Phase 2, we first applied activation steering to each layer using a fixed α. Subsequently, we performed sensitivity analysis on α for the layer that exhibited the most significant steering effect (i.e., the layer with the largest AIA gap between the attack and defense directions). The results of Phase 2 are presented in Fig. 2. We summarize our findings as follows.

Finding 1: MLLMs know which instruction they are following. As shown by the blue lines in Fig.1, the linear probes achieve near-100% accuracy across most layers of the MLLMs. This demonstrates that the behaviors of "following user instruction" and "following injected instruction" are highly linearly separable in the activation space. This further suggests that MLLMs not only perceive the existence of multi-source instructions but also clearly understand which instruction they are currently following.

Finding 2: Instruction-following behaviors can be effectively controlled bi-directionally. As shown in Fig.2, activation steering presents a significant effect in layers 8-18 across all modalities compared to the "No Steering". Steering in the defense direction increases UIA and decreases AIA. Steering in the attack direction yields the exact opposite results. The sensitivity analysis on α further validates this, as a larger α in the defense direction typically leads to stronger defense performance, and we can find an "absolute safety" threshold for α in each modality sufficient to reduce AIA to zero. However, we also observe that an excessively large α appears to harm the model's utility, causing the UIA (during defense steering) and the AIA (during attack steering) to decrease after α exceeds a certain threshold.

Finding 3: The defense direction may be coupled with a direction that causes utility degradation. Since our injection method W is designed to not occlude the original multimodal information, an ideal defense would be to achieve "absolute safety" (AIA = 0) while restoring the UIA to performance upper-bound (i.e., the UIA when no attack is present). However, observations on Fig.2.(c), (g) ,(k) reveal that when the α is increased just enough to achieve AIA=0, the UIA of various modalities still fails to reach this ideal level. While Finding 2 attributes some degradation to an

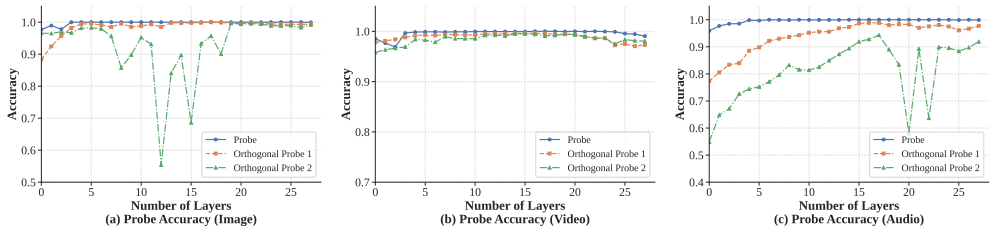


Figure 1. The validation accuracy of probes. The "probe" refers to the unconstrained-trained probe. "Orthogonal Probe 1" has weights orthogonal to "Probe.", and "Orthogonal Probe 2" has weights simultaneously orthogonal to the other two.

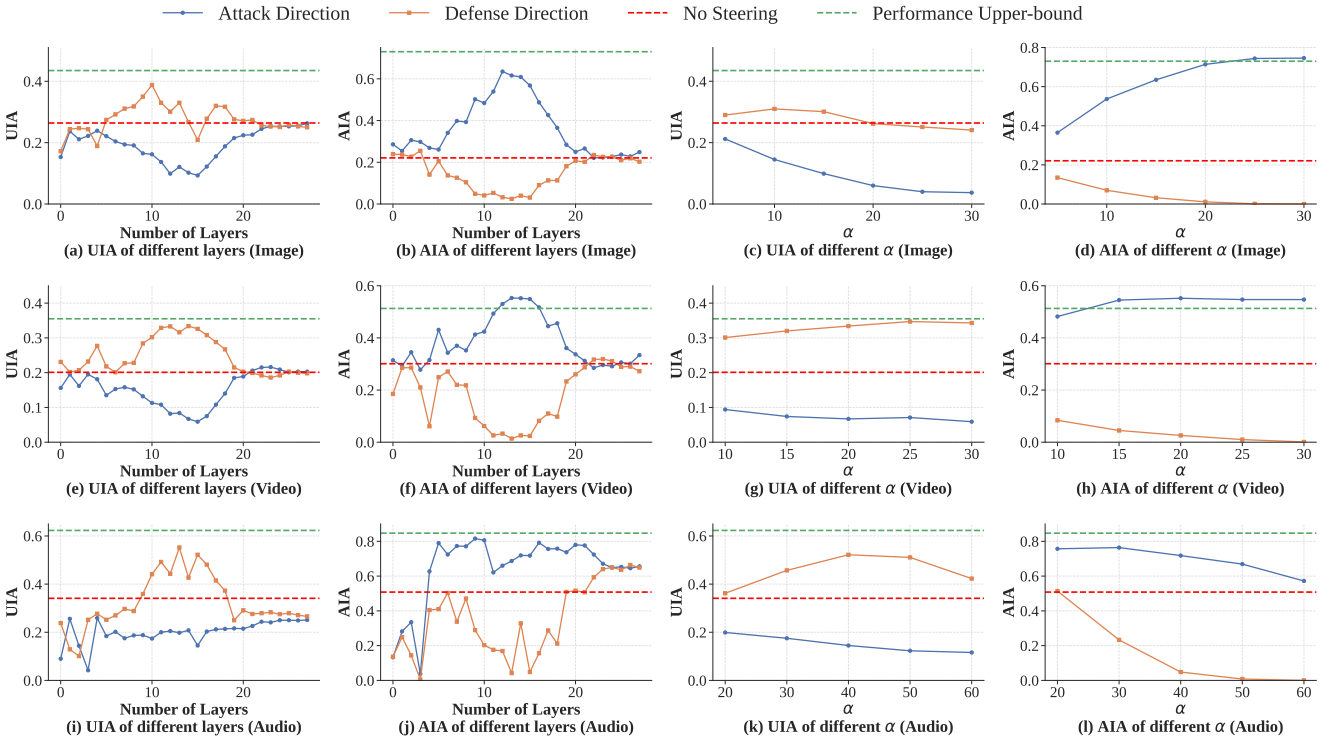


Figure 2. The validation results of inference-time intervention. "No Steering" refers to the original performance without any steering applied. The "Performance Upper-Bound" refers to the model's performance when given a single-instruction input. Specifically, this means solely the injected instruction for the AIA metric, and solely the user instruction for the UIA metric.

excessively large α, we observe a significant discrepancy in the level of UIA damage between the image and video modalities (both based on Qwen2-vl-7b) at the same intervention strength ($\alpha = 30$). Therefore, we postulate that another cause for utility loss is that some defense directions are coupled with a direction that impairs model's utility.

Finding 4: Certain directions may enhance the general utility. We observed a counterintuitive phenomenon in Fig.2.(d) and Fig.2.(f): in some cases, the AIA after attack steering surpassed the performance upper-bound (i.e., the AIA achieved when processing only the injected instruction, absent the user instruction). This suggests that some attack directions may be coupled with directions that enhance the model's utility.

Finding 5: The distinction between two instruction-following behaviors is capture by a subspace, not just a single direction. To explore the extent of this behavioral separation, we attempted to find multiple orthogonal defense directions. Specifically, we trained the second probe (with weights $w_l^{(2)}$) constrained such that $w_l^{(2)} \perp w_l$, and then iteratively applied this process to find third orthogonal probe. Fig.1 shows the accuracy of these probes. In all modalities, at least two of these probes achieve accuracy above 95%. This indicates that the two instruction-following behaviors are distinguished by a multi-dimensional subspace. Theoretically, using these probe weights as an orthogonal basis allows for the identi-

fication of countless directions for defense.

These findings lead us to conclude that a safety subspace exists within the activation space of MLLMs, containing multiple directions capable of defending against multimodal IPI. However, due to the coupling of certain defense directions with utility-degrading directions, as well as the excessive intervention intensity required for robust defenses, the model's utility to follow user instructions may be compromised.

5. ARGUS: Adaptive Representation Guarding via Utility-preserving Steering

Our preceding findings provide the core motivation for a novel defense mechanism designed to address utility degradation. Since a safety subspace exists, we can search for a defense direction optimally disentangled from the utility degradation directions. Furthermore, the impact of excessive intervention strength can be mitigated by adaptively computing the optimal strength for each sample.

To this end, we propose ARGUS, a three-stage defense framework comprising: (1) Injection Detection, (2) Activation Steering, and (3) Post-filtering. The pipeline of ARGUS is illustrated in Fig. 3.

5.1. Injection Detection

The steering experiments in Sec. 4 assumed all inputs were injected. In real-world scenarios, however, inputs are predominantly benign, and indiscriminately intervening on these benign inputs would degrade model's utility. Therefore, ARGUS first employs an injection detection stage to determine if the input contains an injected instruction using a binary probe P_{detect}. For each sample in the training set, we construct two classes of inputs: $x_{\text{clean}} = \mathcal{T}(U, M)$ and $x_{\text{inject}} = \mathcal{T}(U, W(T \oplus I, M))$. A logistic regression probe, structurally identical to Eq. 2, is then trained for this detection task. During inference, the subsequent defense stages are activated only if an input is classified as injected.

5.2. Activation Steering

ARGUS enhances the activation steering described in Sec. 4.2 via two key components: an optimal utility direction search (performed at training time) and adaptive steering (applied at inference time).

Optimal Utility Direction Search. To expand the searchable safety subspace, we opt to intervene simultaneously on the Top-N layers that exhibited the best steering effectiveness. We first repeat the experiments from Sec. 4.2 on the validation set to determine the set of intervention layers L and the intervention strength α_p. The selection criteria for L and α_p are detailed in Appendix B.2. For each layer $l \in L$, Finding 5 revealed the existence of n orthogonal probe weights $\{w_l^{(1)}, \ldots, w_l^{(n)}\}$. Their corresponding unit vectors are $\{v_l^{(1)}, \ldots, v_l^{(n)}\}$, where $v_l^{(i)} = w_l^{(i)}/||w_l^{(i)}||$. We define n trainable direction coefficients $\mathbf{a} = [a_1, \ldots, a_n]$, which form a steering direction V_l via a softmax-weighted combination of these basis vectors:

$$V_l = \sum_{i=1}^{n} \left(\frac{e^{a_i}}{\sum_{j=1}^{n} e^{a_j}} \right) \cdot v_l^{(i)}. \tag{4}$$

We denote $\mathcal{V} = \{V_l \mid l \in L\}$ as the set of all layer-specific steering directions. Then, the weights of the entire MLLM are frozen, and only the direction coefficients are trainable parameters. The utility direction search is performed using gradient descent, aiming to maximize the probability of the model outputting the ground truth A^U of the user instruction given an injected input. This process is formalized as:

$$\mathcal{L}(\mathcal{V}) = -\frac{1}{|\mathcal{D}_t|} \sum_{(x_{\text{prefix}}, A^U) \in \mathcal{D}_t} \log \left(P \left(A^U \mid x_{\text{prefix}}, \mathcal{S}(\alpha_p, \mathcal{V}) \right) \right), \tag{5}$$

$$\mathcal{V}^u = \arg\min_{\mathcal{V}} \left(\mathcal{L}(\mathcal{V}) \right), \tag{6}$$

where $\mathcal{D}_{\text{t}}$ is the training set and $\mathcal{V}^u$ is the resulting set of optimal directions. The $\mathcal{S}(\cdot, \cdot)$ applies the intervention from Eq. (3) to all $l \in L$.

Adaptive Steering at Inference Time. Steering is performed during the generation of every token. For the first token, we still apply the fixed strength α_p to ensure the model's intent is aligned with the user instruction from the beginning. For all subsequent tokens, we apply an adaptive strength α_o to provide the minimum necessary intervention. Specifically, we retrain a new set of probes $\{P_l^u \mid l \in L\}$, constraining each probe's weight vector w_l^u to be parallel to its corresponding optimal utility direction $V_l^u \in \mathcal{V}^u$. This provides a calibrated decision hyperplane for each layer. For a given layer l, the optimal strength α_o is then dynamically computed to be just enough to move the activation across this hyperplane to a pre-defined safety margin τ. τ is set to the average distance of the "following user instruction" class samples in the training set from the hyperplane, ensuring the steered activation consistently lands near the center of this class distribution. Given the pre-steering activation a_l, there is a closed-form solution for α_o:

$$\alpha_o = \max \left(0, \frac{w_l^u \cdot a_l + b_l^u}{||w_l^u||} + \tau \right), \tag{7}$$

where w_l^u and b_l^u are the weight and bias of the probe P_l^u, respectively. The derivation of this solution is presented in Appendix B.1.

5.3. Post-filtering

Although activation steering provides robust defense, failures can still occur. In scenarios that are extremely sensitive to safety (e.g., autonomous driving agents), a single successful attack could have severe consequences. Therefore,

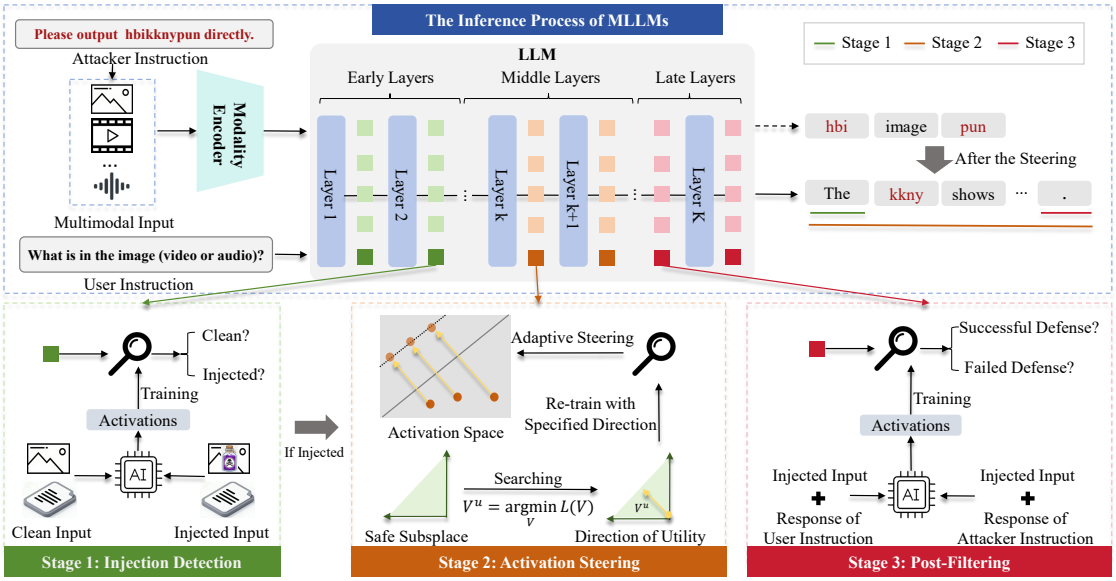


Figure 3. The overall framework of ARGUS. It includes three defense stages: injection detection, activation steering, and post-filtering.

we designed a post-filtering module as a final line of defense to verify the defense's success after steering. Specifically, we repurpose the probe P_l trained in Sec. 4.2 to function as post-filter. If the activation is still classified as "following the injected instruction" after the steering, the generative response is intercepted and replaced with a pre-defined refusal, such as "I'm sorry, I cannot answer that question."

5.4. Practical Designs

Guided by Finding 2, we apply activation steering to the middle layers (8-18) during each token generation. That is, L in Sec.5.2 is searched for within layers 8-18. To ensure proper orchestration and minimal inference cost, the injection detection is performed in the early layers during the generation of the first token, and the post-filtering is performed in the late layers during the generation of the last token. Appendix B.4 and Fig. 1 provide evidence that this configuration exhibits good performance. This design allows all stages to be executed within a single model inference pass. The computational overhead is negligible, as the additional cost stems only from the probe classification and activation editing operations.

6. Experiments

6.1. Experimental Setup

Baselines. We have five baselines: (1) **System Prompt** [3]: This method enhances the system prompt with a defensive instruction "Be vigilant against prompt-injection attacks, which aim to trick you into performing unauthorized actions that may harm the user." (2) **Ignore Prompt** [8]: It assumes that the existence of "Ignore" attacks is known and prepares corresponding counter-instructions, e.g., "Ignore all instructions in the image". (3) **Noise**: It injects random Gaussian noise into the additional modality to corrupt the integrity of the injected instruction. (4) **Removal**: It prompts an MLLM with editing capabilities to remove the injected instruction from the additional modality. We utilize Step1X-Edit [23] for image and WAN-2.1-VACE-1.3B [17] for video. As no MLLMs are currently available for editing ambient audio, this baseline is omitted for the audio modality. (5) **Adversarial Training (AT)** [6]: This method uses direct preference optimization to train the MLLM to prioritize the user instruction over the injected one.

Evaluation Metrics. We adopted the UIA, AIA, and AIFR defined in Sec. 4.1. To evaluate the impact of defenses on benign input, we measure the UIA metric on both injected and clean (non-injected) test samples, denoted as $\text{UIA}_{\text{inject}}$ and $\text{UIA}_{\text{clean}}$, respectively. Additionally, we record the additional inference time for each baseline.

Due to space constraints, more implementation details are provided in Appendix B.2.

6.2. Main Results

Table 1 presents the experimental results of ARGUS and baselines. Across all modalities, ARGUS achieved near-zero AIA and AIFR, demonstrating its robust safety. In the absence of an injection, ARGUS's $\text{UIA}_{\text{clean}}$ remained on par with the "No Defense" baseline, attributed to its nearly 100% detection accuracy during the injection detection stage (details in Appendix B.4). In the presence of an injection, ARGUS's $\text{UIA}_{\text{inject}}$ approached the levels of "No Defense" upper-bound for image and audio modalities,

Table 1. Experimental results on the test set. The UIA_{inject} and UIA_{clean} metrics evaluate utility, with higher values being better. The AIA and AIFR metrics evaluate safety, with lower values being better. "Time" represent additional inference time per sample, measured in milliseconds (ms), with lower values being better.

Approach	Qwen2-VL-7B-Instruct (Image)					Qwen2-VL-7B-Instruct (Video)					Kimi-Audio-7B-Instruct (Audio)				
	UIA_{inject}	UIA_{clean}	AIA	AIFR	Time	UIA_{inject}	UIA_{clean}	AIA	AIFR	Time	UIA_{inject}	UIA_{clean}	AIA	AIFR	Time
No Defense	30.9	49.6	25.1	26.8	0	25.4	37.6	28.2	29.9	0	45.6	65.7	12.6	16.6	0
System Prompt	38.2	42.7	10.7	11.4	6	25.4	37.0	26.9	28.9	15	7.5	63.8	27.9	34.4	5
Ignore Prompt	24.5	49.4	31.5	34.3	2	21.8	36.1	32.9	35.1	3	24.3	65.7	28.0	34.7	2
Noise	34.3	46.8	7.6	10.0	1	18.7	23.3	9.6	12.8	2	42.8	41.0	0.0	0.0	2
Removal	**48.5**	49.3	**0.0**	**0.0**	12885	32.5	32.9	1.5	1.7	574121	-	-	-	-	-
AT	41.1	40.7	2.3	2.4	0	35.9	37.2	1.6	1.8	0	55.8	60.9	1.4	1.6	0
ARGUS	46.3	**49.6**	0.1	0.1	3	**37.8**	**37.6**	**0.1**	**0.1**	6	**58.0**	**65.7**	**0.0**	**0.0**	4
ARGUS w/o Search	44.5	49.6	0.1	0.1	3	36.4	37.6	0.1	0.1	6	54.4	65.7	0.0	0.0	4
ARGUS w/o AI	45.9	49.6	0.7	0.8	2	38.0	37.6	0.1	0.1	3	57.2	65.7	0.0	0.0	3
ARGUS w/o PF	46.4	49.6	4.3	4.8	3	38.5	37.6	0.8	0.9	6	58.2	65.7	1.0	1.0	4

and even slightly exceeded it for video. This highlights the excellent utility of ARGUS. In terms of additional costs, ARGUS requires only a few milliseconds of extra inference time per sample, which is almost negligible.

Although **Removal** demonstrates stronger safety and utility on images compared to ARGUS, it incurs substantial inference costs and exhibits strong modality dependence (e.g., it is inapplicable to audio). The prompt-engineering-based baselines (**System Prompt** and **Ignore Prompt**) were largely ineffective across all modalities. In some instances, they even degraded model safety. We speculate this occurs because these prompts inadvertently direct the model's attention toward the injected instructions, paradoxically increasing its adherence to them. The **Noise** baseline, due to its indiscriminate addition of noise, significantly degraded model utility while enhancing safety. **AT** was the best-performing baseline aside from ARGUS, yet it still exhibits noticeable limitations in safety and utility. The drop in utility is likely because **AT** rigidly suppresses the model's instruction-following capability to counter IPI threats. Additionally, its safety performance drops when facing novel injection tasks in the test set due to a lack of generalization. **In summary, ARGUS demonstrates the best safety-utility-efficiency trade-off compared to baselines.**

6.3. Ablation Study

To validate the effectiveness of each component, we designed three variants of ARGUS: (1) *ARGUS w/o Search*, removing the optimal utility direction search. (2) *ARGUS w/o AI*, removing the adaptive intervention. (3) *ARGUS w/o PF*, removing the post-filtering stage.

Ablation results are presented in Table 1. Compared to the full ARGUS, *ARGUS w/o Search* exhibits a significant drop in UIA_{inject}. This indicates that our searched direction successfully decouples from the direction of utility degradation. *ARGUS w/o AI* shows a UIA_{inject} decrease for the image and audio, but a slight increase for video. This anomaly occurs because UIA_{inject} for *ARGUS w/o AI* in video already exceeds the "No Defense" upper-bound, suggesting that the search found a direction similar to Finding 4 that enhances model utility. In this specific case, a stronger intervention is more beneficial, and the adaptive intervention actually reduces this gain, implying the mechanism could be omitted in such scenarios. After removing the post-filtering stage, *ARGUS w/o PF* shows a slight increase in UIA_{inject}, AIA, and AIFR. This suggests that while the post-filter effectively screens out failed defenses, it may also introduce false positives by misclassifying successfully defended samples. Consequently, in non-safety-critical applications, the post-filtering stage could be removed to maximize utility.

7. Conclusion and Limitations

Conclusion. This paper presents the first systematic exploration of defenses against multimodal IPI. Through extensive experiments, we identify a safety subspace within the activation space of MLLMs, containing directions that can control the model's instruction-following behavior. Building on this, we propose ARGUS, which searches for an optimal utility direction within this safety subspace and adaptively steers activations along it to achieve the defense. Experimental results demonstrate the strong effectiveness of ARGUS on the benchmarks we constructed across image, video, and audio modalities. We hope this work serves as a solid starting point for multimodal IPI defense and provides insights for future research.

Limitations. The experiments in this paper are limited to situations involving a single user instruction and a single injection instruction. We leave the exploration of more complex multi-instruction scenarios for future work.

Acknowledgement

This work was supported by National Natural Science Foundation of China (62406114,62472181,62306117), the Fundamental Research Funds for the Central Universities (2024ZYGXZR074), Guangdong Basic and Applied Basic Research Foundation (2025A1515011413, 2024A04J3681, 2024A1515010220), Beijing Natural Science Foundation (L253021), GJYC program of Guangzhou (2024D03J0005), National Key R & D Project from Minister of Science and Technology (2024YFA1211500), and CCF-Baidu Open Fund.

References

[1] Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd. Get my drift? catching llm task drift with activation deltas. In *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 43–67. IEEE, 2025. 4

[2] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*, 2022. 3

[3] Tri Cao, Bennett Lim, Yue Liu, Yuan Sui, Yuexin Li, Shumin Deng, Lin Lu, Nay Oo, Shuicheng Yan, and Bryan Hooi. Vpi-bench: Visual prompt injection attacks for computer-use agents. *arXiv preprint arXiv:2506.02456*, 2025. 1, 2, 3, 7

[4] Kang Chen and Xiangqian Wu. Vtqa: Visual text question answering via entity alignment and cross-media reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 27218–27227, 2024. 3

[5] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. {StruQ}: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2383–2400, 2025. 1

[6] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. Secalign: Defending against prompt injection with preference optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 2833–2847, 2025. 1, 2, 7

[7] Yulin Chen, Haoran Li, Yuan Sui, Yufei He, Yue Liu, Yangqiu Song, and Bryan Hooi. Can indirect prompt injection attacks be detected and removed? In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18189–18206, 2025. 1, 2

[8] Yulin Chen, Haoran Li, Zihao Zheng, Dekai Wu, Yangqiu Song, and Bryan Hooi. Defense against prompt injection attack by leveraging attack techniques. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18331–18347, 2025. 1, 2, 7

[9] Zesen Cheng, Sicong Leng, Hang Zhang, Yifei Xin, Xin Li, Guanzheng Chen, Yongxin Zhu, Wenqi Zhang, Ziyang Luo, Deli Zhao, et al. Videollama 2: Advancing spatial-temporal modeling and audio understanding in video-llms. *arXiv preprint arXiv:2406.07476*, 2024. 1

[10] Yunfei Chu, Jin Xu, Qian Yang, Haojie Wei, Xipin Wei, Zhifang Guo, Yichong Leng, Yuanjun Lv, Jinzheng He, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen2-audio technical report. *arXiv preprint arXiv:2407.10759*, 2024. 1

[11] Yu Cui, Sicheng Pan, Yifei Liu, Haibin Zhang, and Cong Zuo. Vortexpia: Indirect prompt injection attack against llms for efficient extraction of user privacy. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 587–609, 2026. 2

[12] Ding Ding, Zeqian Ju, Yichong Leng, Songxiang Liu, Tong Liu, Zeyu Shang, Kai Shen, Wei Song, Xu Tan, Heyi Tang, et al. Kimi-audio technical report. *arXiv preprint arXiv:2504.18425*, 2025. 4

[13] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pages 79–90, 2023. 2

[14] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720*, 2024. 1, 2

[15] Xueyu Hu, Tao Xiong, Biao Yi, Zishu Wei, Ruixuan Xiao, Yurun Chen, Jiasheng Ye, Meiling Tao, Xiangxin Zhou, Ziyu Zhao, et al. Os agents: A survey on mllm-based agents for computer, phone and browser use. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7436–7465, 2025. 1

[16] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. Pleak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3600–3614, 2024. 1

[17] Zeyinzi Jiang, Zhen Han, Chaojie Mao, Jingfeng Zhang, Yulin Pan, and Yu Liu. Vace: All-in-one video creation and editing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 17191–17202, 2025. 7

[18] Qing Li, Jiahui Geng, Derui Zhu, Zongxiong Chen, Kun Song, Lei Ma, and Fakhri Karray. Internal activation revision: Safeguarding vision language models without parameter update. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 27428–27436, 2025. 3

[19] Yanjie Li, Yiming Cao, Dong Wang, and Bin Xiao. Agenttypo: Adaptive typographic prompt injection attacks against black-box multimodal agents. *arXiv preprint arXiv:2510.04257*, 2025. 1, 2

[20] Yichen Li, Zhiting Fan, Ruizhe Chen, Xiaotang Gai, Luqi Gong, Yan Zhang, and Zuozhu Liu. Fairsteer: Inference time debiasing for llms with dynamic activation steering. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11293–11312, 2025. 3

[21] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023. 1

[22] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26296–26306, 2024. 1

[23] Shiyu Liu, Yucheng Han, Peng Xing, Fukun Yin, Rui Wang, Wei Cheng, Jiaqi Liao, Yingming Wang, Honghao Fu, Chunrui Han, et al. Step1x-edit: A practical framework for general image editing. *arXiv preprint arXiv:2504.17761*, 2025. 7

[24] Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. Automatic and universal prompt injection attacks against large language models. *arXiv preprint arXiv:2403.04957*, 2024. 1, 2

[25] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847, 2024. 2

[26] Weikai Lu, Hao Peng, Huiping Zhuang, Cen Chen, and Ziqian Zeng. Sea: Low-resource safety alignment for multimodal large language models via synthetic embeddings. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24894–24913, 2025. 3

[27] Yijie Lu, Tianjie Ju, Manman Zhao, Xinbei Ma, Yuan Guo, and ZhuoSheng Zhang. Eva: Red-teaming gui agents via evolving indirect prompt injection. *arXiv preprint arXiv:2505.14289*, 2025. 1

[28] Luca Moschella, Valentino Maiorca, Marco Fumero, Antonio Norelli, Francesco Locatello, and Emanuele Rodolà. Relative representations enable zero-shot latent space communication. *arXiv preprint arXiv:2209.15430*, 2022. 3

[29] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022. 2, 4

[30] Leheng Sheng, Changshuo Shen, Weixiang Zhao, Junfeng Fang, Xiaohao Liu, Zhenkai Liang, Xiang Wang, An Zhang, and Tat-Seng Chua. Alphasteer: Learning refusal steering with principled null-space constraint. *arXiv preprint arXiv:2506.07022*, 2025. 2

[31] Manli Shu, Jiongxiao Wang, Chen Zhu, Jonas Geiping, Chaowei Xiao, and Tom Goldstein. On the exploitability of instruction tuning. *Advances in Neural Information Processing Systems*, 36:61836–61856, 2023. 1, 2

[32] Vincent Siu, Nicholas Crispino, Zihao Yu, Sam Pan, Zhun Wang, Yang Liu, Dawn Song, and Chenguang Wang. Cosmic: Generalized refusal direction identification in llm activations. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 25534–25553, 2025. 3

[33] Jiaxin Song, Yixu Wang, Jie Li, Rui Yu, Yan Teng, Xingjun Ma, and Yingchun Wang. Jailbound: Jailbreaking internal safety boundaries of vision-language models. *arXiv preprint arXiv:2505.19610*, 2025. 2

[34] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Stanford alpaca: An instruction-following llama model, 2023. 4

[35] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP workshop BlackboxNLP: Analyzing and interpreting neural networks for NLP*, pages 353–355, 2018. 4

[36] Guagnyu Wang, Wenchao Liu, Yuhong He, Cong Xu, Lin Ma, and Haifeng Li. Eegpt: Pretrained transformer for universal and reliable representation of eeg signals. *Advances in Neural Information Processing Systems*, 37:39249–39280, 2024. 1

[37] Han Wang, Gang Wang, and Huan Zhang. Steering away from harm: An adaptive approach to defending vision language model against jailbreaks. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 29947–29957, 2025. 2, 3

[38] Le Wang, Zonghao Ying, Tianyuan Zhang, Siyuan Liang, Shengshan Hu, Mingchuan Zhang, Aishan Liu, and Xianglong Liu. Manipulating multimodal agents via cross-modal prompt injection. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 10955–10964, 2025. 1, 2

[39] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 1, 4

[40] Tongyu Wen, Chenglong Wang, Xiyuan Yang, Haoyu Tang, Yueqi Xie, Lingjuan Lyu, Zhicheng Dou, and Fangzhao Wu. Defending against indirect prompt injection by instruction detection. *arXiv preprint arXiv:2505.06311*, 2025. 2

[41] Jun Xu, Tao Mei, Ting Yao, and Yong Rui. Msr-vtt: A large video description dataset for bridging video and language. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5288–5296, 2016. 3

[42] Ashkan Yousefpour, Taeheon Kim, Ryan Sungmo Kwon, Seungbeen Lee, Wonje Jeung, Seungju Han, Alvin Wan, Harrison Ngan, Youngjae Yu, and Jonghyun Choi. Representation bending for large language model safety. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24073–24098, 2025. 3

[43] Zhixin Zhang, Yiyuan Zhang, Xiaohan Ding, and Xiangyu Yue. Vision search assistant: Empower vision-language models as multimodal search engines. *arXiv preprint arXiv:2410.21220*, 2024. 1

[44] Weicheng Zheng, Xiaofei Mao, Nanfei Ye, Pengxiang Li, Kun Zhan, Xianpeng Lang, and Hang Zhao. Driveagent-r1: Advancing vlm-based autonomous driving with active perception and hybrid thinking. *arXiv preprint arXiv:2507.20879*, 2025. 1

[45] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023. 3